

Netiesinis programavimas

Lukaš Patrik Magalinski

Lapkritis 2024

Laboratorinio darbo formulavimas: Apsirašyti tikslo funkciją $f(X)$, lygybinio ir nelygybinių apribojimų funkcijas $g_i(X)$ ir $h_j(X)$ taip, kad optimizavimo uždavinys būtų formuluojamas $\min f(X)$, $g_i(X)=0$, $h_i(X) \leq 0$. Apskaičiuoti funkcijų $f(X)$, $g_i(X)$, $h_j(X)$ reikšmes taškuose $X_0=(0,0,0)$, $X_1=(1,1,1)$, $X_m=(a/10,b/10,c/10)$, kur $a=1$, $b=0$, $c=4$. Taip pat reikia minimizuoti baudos funkciją praeitame laboratoriniame darbe sukurtu optimizavimo be apribojimų algoritmu, sprendžiant optimizavimo uždavinių seką su mažėjančia parametro r seka.

Laboratorinio darbo tikslas: Rasti kokie turėtų būti stačiakampio gretasienio matmenys, kad vienetiniam paviršiaus plotui jos tūris būtų maksimalus.

1) Uždavinys:

Turime stačiakampio gretasienio formos dėžę su vienetiniu paviršiaus plotu. Kokie turėtų būti matmenys, kad dėžės tūris būtų maksimalus?

Problemą išspręsti pasinaudosime netiesinio programavimo teorija, t.y. apibūdinkime tikslo ir baudos funkcijas. Naudosime deformuojamo simplekso metodą skirtinguose pradiniuose taškuose, kad rasti sprendinius.

2) Tikslo funkcija:

Turime tūrį $V = xyz$, kur x, y, z - dėžės matmenys

Turime vienetinį paviršiaus plotą $S = 2xy + 2xz + 2yz = 1$

Tikslo funkcija:

$$V(x, y, z) = -xyz$$

3) Baudos funkcija $B(X, r)$:

Apibrėžkime nelygybines apribojimo funkcijas:

♦ $h_1(x, y, z) = -x$

♦ $h_2(x, y, z) = -y$

♦ $h_3(x, y, z) = -z$

Lygybinio apribojimo funkcija:

$$g_1(x, y, z) = 2xy + 2xz + 2yz - 1$$

Iš to galima sukonstruoti baudos funkciją $B(X, r)$:

$$B(X, r) = V(X) + \frac{1}{r} b(X)$$

$$b(X) = \max(0, h_1(X))^2 + \max(0, h_2(X))^2 + \max(0, h_3(X))^2 + g_1(X)^2$$

$$b(x, y, z) = \max(0, -x)^2 + \max(0, -y)^2 + \max(0, -z)^2 + g_1(x, y, z)^2$$

$$B(x, y, z, r) = V(x, y, z) + \frac{1}{r} (\max(0, -x)^2 + \max(0, -y)^2 + \max(0, -z)^2 + g_1(x, y, z)^2)$$

Apskaičiuokime reikšmes skirtinguose pradiniuose taškuose:

- ♦ $X_0 = (0,0,0)$:
 - $V(0,0,0) = 0$
 - $g_1(0,0,0) = -1$
 - $h_1(0,0,0) = 0$
 - $h_2(0,0,0) = 0$
 - $h_3(0,0,0) = 0$
- ♦ $X_1 = (1,1,1)$:
 - $V(1,1,1) = 1$
 - $g_1(1,1,1) = 5$
 - $h_1(1,1,1) = -1$
 - $h_2(1,1,1) = -1$
 - $h_3(1,1,1) = -1$

$X_m = \left(\frac{a}{10}, \frac{b}{10}, \frac{c}{10}\right)$, kur a, b ir c yra studento pažymėjimo numerio 3 paskutinės skaitmenys. Mano studento

pažymėjimo numeris [redacted]. Iš čia: $X_m = \left(\frac{1}{10}, \frac{0}{10}, \frac{4}{10}\right)$, $X_m = (0.1, 0, 0.4)$.

- ♦ $X_m = (0.1,0,0.4)$:
 - $V(0.1,0,0.4) = 0$
 - $g_1(0.1,0,0.4) = -0.92$
 - $h_1(0.1,0,0.4) = -0.1$
 - $h_2(0.1,0,0.4) = 0$
 - $h_3(0.1,0,0.4) = -0.4$

4) Baudos metodas ir sprendimo algoritmas:

Šiame laboratoriniame darbe bus analizuojama, kaip taikant baudos metodą galima spręsti optimizavimo uždavinius. Pirmiausia pradedama nuo taško X_1 , esančio leistinoje srityje. Naudojant optimizavimo algoritmą, šiuo atveju, kaip jau buvo parašyta: deformuojamo simplekso metodą, iteratyviai ieškoma tikslo funkcijos lokalaus minimumo. Pagrindinės iteracijų nutraukimo sąlygos:

- 1) Simplekso metode – jei pasiekiamas norimas tikslumas epsilon (atstumas tarp blogiausio ir geriausio taško) arba pasiektas maksimalus iteracijų kiekis, kuris yra 250.
- 2) Pagrindinio while ciklo sustojimo sąlyga: leidžia algoritmui tęsti iteracijas tol, kol baudos funkcija mažėja arba bet kuris kintamasis pažeidžia nelygybinius apribojimus (kai $x, y, z < 0$).

Deformuojamo simplekso (Nelder–Mead) metodas yra netiesinės optimizacijos algoritmas, kuriame pradinis tetraedrinis taškų rinkinys (simpleksas) iteratyviai atnaujinamas, siekiant rasti funkcijos minimumą. Kiekviename žingsnyje taškai surūšiuojami pagal tikslinės funkcijos reikšmes, o blogiausias taškas atspindimas per likusių trijų taškų centroidą. Jei naujasis taškas pagerina rezultatą, jis papildomai išplečiamas. Priešingu atveju atliekama kontrakcija, o jei reikia – visi taškai sumažinami artėjant link geriausiojo. Algoritmas tęsiasi tol, kol simplekso „perimetras“ sumažėja iki iš anksto nustatytos ribos arba pasiekiamas maksimalus iteracijų skaičius. Norint atsižvelgti į apribojimus (pvz., $x, y, z \geq 0$), naudojama baudos funkcija. Ji padidina tikslią funkciją, jei sprendinyje pažeidžiamos apribojimo sąlygos, taip verčiant simpleksą konverguoti leistinoje srityje.

Visus skaičiavimus atliksime iki tikslumo $\varepsilon = 0.0001$, o deformuojamo simplekso metodo parametrai bus:
 $\alpha = 1$, $\gamma = 2$, $\beta = 0.5$, $\tau = 0.5$.

5) Netiesinio programavimo teorijos taikymas, rezultatai:

Parašiau Python programą (žr. priedą), kurioje taikoma netiesinio programavimo teorija. Pradinis taškas pasirenkamas iš X_0 , X_1 arba X_m . Iš šio pradinio taško pradedame greičiausiojo nusileidimo metodą. Kiekvieno vėlesnio uždavinio pradinis taškas tampa ankstesnio uždavinio sprendiniu. Pradinis baudos koeficientas r yra 10. Be to, pabandysime atkreipti dėmesį į rezultatų skirtumą, jei baudos koeficientą r padalintas iš skirtingų q (baudos daliklį toliau žymėsime raide q) reikšmių: $q = 2$, $q = 5$, $q = 20$. Pats baudos koeficientas r išliks pastovus ir bus lygus 10.

5.1) Pradinis taškas $X_0 = (0, 0, 0)$:

5.1.1) Baudos koeficiento r daliklis $q = 2$:

Baudos funkcijos minimizavimas						
$\frac{r}{2}$	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
	0	10	(0.00000000, 0.00000000, 0.00000000)	0.10000000	0.100000	0.000000
	1	5.0	(0.52549969, 0.52549579, 0.52549430)	0.04314914	-0.058816	-0.145114
	2	2.5	(0.46363664, 0.46364865, 0.46363621)	0.01679392	-0.066077	-0.099665
	3	1.25	(0.43512020, 0.43511860, 0.43511819)	0.00739527	-0.067590	-0.082380
	4	0.625	(0.42147663, 0.42147227, 0.42148008)	0.00346937	-0.067933	-0.074872
	5	0.3125	(0.41481535, 0.41480596, 0.41480905)	0.00168010	-0.068015	-0.071375
	6	0.15625	(0.41151817, 0.41151245, 0.41151870)	0.00082687	-0.068035	-0.069689
	7	0.078125	(0.40987895, 0.40987927, 0.40988011)	0.00041031	-0.068040	-0.068860
	8	0.0390625	(0.40906545, 0.40905439, 0.40906909)	0.00020430	-0.068041	-0.068450
	...	...	...	...	...	...
	26	1.490e-07	(0.40824141, 0.40823713, 0.40826633)	6.587e-10	-0.068041	-0.068041
	27	7.450e-08	(0.40824371, 0.40823818, 0.40826299)	2.672e-10	-0.068041	-0.068041
	28	3.725e-08	(0.40824939, 0.40823748, 0.40825801)	2.520e-10	-0.068041	-0.068041
	29	1.862e-08	(0.40824239, 0.40824108, 0.40826140)	1.643e-10	-0.068041	-0.068041
	30	9.313e-09	(0.40825243, 0.40823895, 0.40825349)	1.820-11	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{2}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	31	(0.4082524, 0.4082389, 0.4082534)	-0.068041381	4367

5.1.2) Baudos koeficiento r daliklis $q = 5$:

Baudos funkcijos minimizavimas						
$\frac{r}{5}$	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
	0	10	(0.00000000, 0.00000000, 0.00000000)	0.1	0.100000	0.000000
	1	2.0	(0.52549969, 0.52549579, 0.52549430)	0.04314913644	0.070632	-0.145114
	2	0.4	(0.42960831, 0.42961847, 0.42960900)	0.00576721328	-0.050456	-0.079292
	3	0.08	(0.41244236, 0.41242833, 0.41243837)	0.00106321638	-0.064841	-0.070157
	4	0.016	(0.40909018, 0.40908308, 0.40907477)	0.00020928727	-0.067413	-0.068459
	5	0.0032	(0.40840929, 0.40841570, 0.40841970)	4.165e-05	-0.06791	-0.068125
	...	...	...	...	...	...
	10	1.0243e-06	(0.40823793, 0.40825819, 0.40824891)	1.325e-08	-0.068041	-0.068041
	11	2.048e-07	(0.40825196, 0.40824965, 0.40824328)	1.456e-09	-0.068041	-0.068041
	12	4.096e-08	(0.40821657, 0.40829559, 0.40823271)	3.205e-11	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{5}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	13	(0.4082165, 0.4082955, 0.4082327)	-0.068041381	1906

5.1.3) Baudos koeficiento r daliklis $q = 20$:

Baudos funkcijos minimizavimas						
	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
$\frac{r}{20}$	0	10	(0.00000000, 0.00000000, 0.00000000)	0.1	0.100000	0.000000
	1	0.5	(0.52549969, 0.52549579, 0.52549430)	0.04314913644	0.717869	-0.145114
	2	0.025	(0.41349287, 0.41349560, 0.41348145)	0.00133580044	-0.043980	-0.070696
	3	0.00125	(0.40850850, 0.40850273, 0.40851544)	6.52360393e-05	-0.066867	-0.068172
	4	6.25e-05	(0.40825823, 0.40826353, 0.40826225)	3.26791542e-06	-0.067983	-0.068048
	5	3.125e-06	(0.40825469, 0.40825367, 0.40823847)	1.63615782e-07	-0.068038	-0.068042
	6	1.5625e-07	(0.40824055, 0.40825738, 0.40824704)	7.76280343e-09	-0.068041	-0.068041
	7	7.8125e-09	(0.40824638, 0.40825688, 0.40824162)	3.757797e-10	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{20}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	8	(0.40827956, 0.4082311, 0.40823418)	-0.0680413813	1374

5.2) Pradinis taškas $X_1 = (1, 1, 1)$:

5.2.1) Baudos koeficiento r daliklis $q = 2$:

Baudos funkcijos minimizavimas						
	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
$\frac{r}{2}$	0	10	(1.00000000, 1.00000000, 1.00000000)	2.5	1.500000	-1.000000
	1	5.0	(0.52549427, 0.52550243, 0.52549497)	0.04314965802	-0.058815	-0.145115
	2	2.5	(0.46364048, 0.46363667, 0.46364837)	0.01679478410	-0.066077	-0.099666
	3	1.25	(0.43511988, 0.43510998, 0.43512325)	0.00739453785	-0.067591	-0.082380
	4	0.625	(0.42147670, 0.42146944, 0.42148386)	0.00346955706	-0.067933	-0.074872
	5	0.3125	(0.41481418, 0.41480920, 0.41480994)	0.00168060426	-0.068015	-0.071376
	6	0.15625	(0.41150700, 0.41152243, 0.41152016)	0.000826911662	-0.068035	-0.069689
	7	0.078125	(0.40988273, 0.40987713, 0.40987862)	0.00041033702	-0.068040	-0.068860
	8	0.0390625	(0.40905953, 0.40906710, 0.40906144)	0.00020415775	-0.068041	-0.068449
	...	...	...	...	...	...
	23	1.19209e-06	(0.40824740, 0.40825474, 0.40824280)	5.92013150e-09	-0.068041	-0.068041
	24	5.96046e-07	(0.40824238, 0.40824767, 0.40825486)	3.45721702e-09	-0.068041	-0.068041
	25	2.98023e-07	(0.40823512, 0.40825828, 0.40825149)	1.59428225e-09	-0.068041	-0.068041
	26	1.49011e-07	(0.40823398, 0.40825533, 0.40825557)	5.54705311e-10	-0.068041	-0.068041
	27	7.45058e-08	(0.40824135, 0.40824036, 0.40826317)	2.41737347e-10	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{2}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	28	(0.40824594, 0.4082398, 0.40825906)	-0.0680413822	3986

5.2.2) Baudos koeficiento r daliklis $q = 5$:

Baudos funkcijos minimizavimas						
$\frac{r}{5}$	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
	0	10	(1.00000000, 1.00000000, 1.00000000)	2.5	1.500000	-1.000000
	1	2.0	(0.52549427, 0.52550243, 0.52549497)	0.04314965802	0.070634	-0.145115
	2	0.4	(0.42961608, 0.42961137, 0.42961641)	0.00576870377	-0.050450	-0.079293
	3	0.08	(0.41243646, 0.41244296, 0.41242846)	0.00106301547	-0.064842	-0.070157
	4	0.016	(0.40908204, 0.40909011, 0.40907584)	0.00020928181	-0.067413	-0.068459
	5	0.0032	(0.40908204, 0.40909011, 0.40907584)	4.16493523e-05	-0.067916	-0.068125
	...	...	...	...	...	...
	12	4.09600e-08	(0.40820393, 0.40833719, 0.40820376)	2.16972654e-10	-0.068041	-0.068041
	13	8.19201e-09	(0.40824524, 0.40824958, 0.40825005)	3.94520581e-11	-0.068041	-0.068041
	14	1.63840e-09	(0.40824874, 0.40824007, 0.40825606)	3.16052395e-12	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{5}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	15	(0.4082487, 0.4082400, 0.40825605)	-0.0680413817	2255

5.2.3) Baudos koeficiento r daliklis $q = 20$:

Baudos funkcijos minimizavimas						
$\frac{r}{20}$	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
	0	10	(1.00000000, 1.00000000, 1.00000000)	2.5	1.500000	-1.000000
	1	0.5	(0.52549427, 0.52550243, 0.52549497)	0.04314965802	0.717879	-0.145115
	2	0.025	(0.41349768, 0.41348158, 0.41348994)	0.00133567564	-0.043982	-0.070696
	3	0.00125	(0.40850903, 0.40851234, 0.40850521)	6.52208553e-05	-0.066867	-0.068172
	4	6.25e-05	(0.40826650, 0.40825890, 0.40825861)	3.26786967e-06	-0.067983	-0.068048
	5	3.125e-06	(0.40825573, 0.40824708, 0.40824402)	1.63785786e-07	-0.068038	-0.068042
	6	1.5625e-07	(0.40825033, 0.40825402, 0.40824062)	3.26786967e-06	-0.068041	-0.068041
	7	7.81250e-09	(0.40824735, 0.40824838, 0.40824914)	2.72457804e-10	-0.068041	-0.068041
	8	3.90625e-10	(0.40824148, 0.40827254, 0.40823086)	2.38747351e-13	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{20}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	9	(0.40825641, 0.40826092, 0.4082275)	-0.0680413816	1527

5.3) Pradinis taškas $X_1 = (0.1, 0, 0.4)$:

5.3.1) Baudos koeficiento r daliklis $q = 2$:

Baudos funkcijos minimizavimas						
	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
$\frac{r}{2}$	0	10	(0.10000000, 0.00000000, 0.40000000)	0.08463999999	0.084640	-0.000000
	1	5.0	(0.52550214, 0.52549086, 0.52549223)	0.04314787973	-0.058817	-0.145113
	2	2.5	(0.46364477, 0.46364082, 0.46364085)	0.01679498043	-0.066076	-0.099666
	3	1.25	(0.43511965, 0.43511413, 0.43512025)	0.00739471075	-0.067590	-0.082380
	4	0.625	(0.42147757, 0.42147205, 0.42148109)	0.00346968302	-0.067933	-0.074872
	5	0.3125	(0.41480960, 0.41481236, 0.41481102)	0.00168054597	-0.068015	-0.071376
	6	0.15625	(0.41152469, 0.41151708, 0.41150922)	0.00082714804	-0.068035	-0.069689
	7	0.078125	(0.40987760, 0.40988687, 0.40987262)	0.00041010212	-0.068040	-0.068860
	8	0.0390625	(0.40906008, 0.40905956, 0.40906926)	0.00020429498	-0.068041	-0.068450
	...	...	...	...	...	...
	26	1.4901e-07	(0.40823248, 0.40822446, 0.40828795)	9.4031007e-10,	-0.068041	-0.068041
	27	7.4505e-08	(0.40823495, 0.40822439, 0.40828554)	4.7999866e-10	-0.068041	-0.068041
	28	3.7252e-08	(0.40823001, 0.40823049, 0.40828438)	3.3085177e-10	-0.068041	-0.068041
	29	1.8626e-08	(0.40823851, 0.40822353, 0.40828283)	1.1061358e-10	-0.068041	-0.068041
	30	9.3132e-09	(0.40823842, 0.40823319, 0.40827326)	1.6327101e-11	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{2}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	31	(0.4082384, 0.40823319, 0.40827325)	-0.0680413817	4457

5.3.2) Baudos koeficiento r daliklis $q = 5$:

Baudos funkcijos minimizavimas						
	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
$\frac{r}{5}$	0	10	(0.10000000, 0.00000000, 0.40000000)	0.08463999999	0.084640	-0.000000
	1	2.0	(0.52550214, 0.52549086, 0.52549223)	0.04314787973	0.070627	-0.145113
	2	0.4	(0.42961544, 0.42961255, 0.42961304)	0.00576818230	-0.050452	-0.079293
	3	0.08	(0.41243418, 0.41242935, 0.41244411)	0.00106297332	-0.064842	-0.070157
	4	0.016	(0.40908603, 0.40908537, 0.40907603)	0.00020918664	-0.067413	-0.068459
	5	0.0032	(0.40841882, 0.40840764, 0.40841866)	4.17243720e-05	-0.067916	-0.068125
	...	...	...	...	...	...
	10	5.12000e-06	(0.40825376, 0.40824939, 0.40824252)	6.58242181e-08	-0.068041	-0.068042
	11	1.02400e-06	(0.40825078, 0.40825072, 0.40824353)	1.39754735e-08	-0.068041	-0.068041
	12	2.04800e-07	(0.40825721, 0.40823346, 0.40825423)	1.56422274e-09	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{5}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	12	(0.4082269, 0.40821692, 0.40830103)	-0.0680413832	1772

5.3.3) Baudos koeficiento r daliklis $q = 20$:

Baudos funkcijos minimizavimas						
	Iteracija	r	(x, y, z)	Baudos įvertis $1/r * b(X)$	$B(X, r)$	$V(X)$
$\frac{r}{20}$	0	10	(0.10000000, 0.00000000, 0.40000000)	0.08463999999	0.084640	-0.000000
	1	0.5	(0.52550214, 0.52549086, 0.52549223)	0.04314787973	0.717845	-0.145113
	2	0.025	(0.41349145, 0.41348989, 0.41348688)	0.00133550689	-0.043986	-0.070696
	3	0.00125	(0.40851215, 0.40851297, 0.40850145)	6.52212247e-05	-0.066867	-0.068172
	4	6.25e-05	(0.40825973, 0.40826488, 0.40825929)	3.24998837e-06	-0.067983	-0.068048
	5	3.125e-06	(0.40825059, 0.40824672, 0.40824953)	1.64515328e-07	-0.068038	-0.068042
	6	1.5625e-07	(0.40822962, 0.40826417, 0.40825118)	7.05977026e-09	-0.068041	-0.068041
	7	7.81250e-09	(0.40824484, 0.40824863, 0.40825140)	4.90180895e-10	-0.068041	-0.068041

Baudos funkcijos minimizavimo rezultatai				
$\frac{r}{20}$	Iteracijų skaičius	X_{min}	$V(X)$	$B(X, r)$ iškvietimų
	8	(0.40823563, 0.40827761, 0.4082316)	-0.0680413813	1327

6) Išvados:

Apibendrinant galima būtų teigti, jog iš visų taškų metodus tinkamai atliko savo darbą – surado funkcijos minimumą, o iteracijų skaičiui ir skaičiuotų funkcijų kiekiui įtaką turi r ir q parametrai: kuo mažesnis r , tuo didesnę įtaką funkcija $b(x)$ turi funkcijai $B(X, r)$, kas matosi ir iš formulės $B(X, r) = f(X) + 1/r * b(X)$. Kuo r greičiau mažėja, t.y. kuo q didesnis, tuo kiekvienoj iteracijoje funkcija $b(X)$ turi vis didesnę įtaką funkcijai $B(X, r)$, dėl ko greičiau atrandamas minimumas. Tačiau pradedant su per mažu r arba didesniu q , funkcija $b(X)$ turi per didelę įtaką funkcijai $B(X, r)$, dėl ko tikslo funkcijos $f(X)$ įtaka tampa per maža ir sumažėja metodo tikslumas. Naudojant tuos pačius parametrus q ir r iš skirtingų pradžios taškų metodui prireikė beveik vienodai iteracijų bei panašiai funkcijų skaičiavimų, o gauti X_{min} ir $f(X_{min})$ labai panašūs. Taip pat buvo naudotas deformuojamo simplekso metodas baudos funkcijos minimizavimui, kurio parametrų šiame laboratoriniame darbe nekeičiau ir nenagrinėjau, tačiau iš praeito laboratorinio darbo rezultatų žinoma, jog metodo parametrai taip pat turi didelę įtaką minimumui rasti, juos pakoregavus veikiausiai būtų galima minimumą rasti dar greičiau, taip pat glaima pakeisti pagrindinio ciklo sustojimo sąlygą, nes ji taip pat turi nemažą įtaką minimumui rasti.

Apibendrinant, pradinė užduotis – stačiakampio gretasienio formos dėžės su vienetiniu paviršiaus plotu tūrio maksimizavimas – sėkmingai išsprendžiama naudojant deformuojamo simplekso metodą ir baudos funkciją. Tyrimai patvirtina, jog optimalūs matmenys (maždaug $x \approx y \approx z \approx 0.408$) padidina dėžės tūrį. Taip pat nustatyta, kad tinkamai parinkus baudos koeficientą ir jo mažinimo tempą, metodas stabiliai konverguoja link sprendinio, artimo kubo formai. Pastebėta, kad pradinių taškų pasirinkimas ir parametrų reikšmės daro įtaką algoritmo darbui, tačiau teisingai suderinus parametrus galima efektyviai rasti optimalų dėžės sprendinį, nepriklausomai nuo pradinio spėjimo

7) Priedas:

```
import math

class Vector(object):
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def length(self):
        return math.sqrt(self.x ** 2 + self.y ** 2 + self.z ** 2)

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y, self.z +
other.z)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y, self.z -
other.z)

    def __mul__(self, other: float):
        return Vector(self.x * other, self.y * other, self.z * other)

    def __rmul__(self, other: float):
        return self.__mul__(other)

    def __truediv__(self, other: float):
        ratio = 1.0 / other
        return self * ratio

    def __str__(self):
        return "({0}, {1}, {2})".format(self.x, self.y, self.z)

    def __repr__(self):
        return f"Vector({self.x}, {self.y}, {self.z})"

def simplexMethod(f, X0, reflectionRate: float = 1.0, expansionRate:
float = 2.0, contractionRate: float = 0.5, tau: float = 1, epsilon: float
= 0.0001, maxIterations: int = 250):
    def contract():
        nonlocal worst, fWorst, good1, fGood1, good2, fGood2, mid

        contractedPoint = mid + contractionRate * (worst - mid)
        fContr = f(contractedPoint)

        if fContr < fWorst:
            worst, fWorst = contractedPoint, fContr
        else:
```

```

        good1 = (good1 + best) / 2
        fGood1 = f(good1)

        good2 = (good2 + best) / 2
        fGood2 = f(good2)

        worst = (worst + best) / 2
        fWorst = f(worst)

    delta1 = (2 * math.sqrt(2) / 3.0) * tau
    delta2 = (math.sqrt(2) / 6) * tau

    X1 = Vector(X0.x + delta2, X0.y + delta1, X0.z + delta1)
    X2 = Vector(X0.x + delta1, X0.y + delta2, X0.z + delta1)
    X3 = Vector(X0.x + delta1, X0.y + delta1, X0.z + delta2)
    #print(f"x1: {X1}, x2: {X2}, x3: {X3}")
    simplex = [ (X0, f(X0)), (X1, f(X1)), (X2, f(X2)), (X3, f(X3)) ]
    iterations = 0

    while iterations < maxIterations:
        iterations += 1
        #print(iterations)

        points = sorted(simplex, key=lambda x: x[1])

        best, fBest = points[0][0], points[0][1]
        good1, fGood1 = points[1][0], points[1][1]
        good2, fGood2 = points[2][0], points[2][1]
        worst, fWorst = points[3][0], points[3][1]

        #print(best, good1, good2, worst)

        mid = (best + good1 + good2) / 3.0
        refl = mid + reflectionRate * (mid - worst)
        fRefl = f(refl)
        if fRefl < fBest:
            exp = mid + expansionRate * (refl - mid)
            fExp = f(exp)
            if fExp < fRefl:
                worst, fWorst = exp, fExp
            else:
                worst, fWorst = refl, fRefl
        elif fRefl < fGood2 and fRefl > fBest:
            worst, fWorst = refl, fRefl
        elif fRefl < fWorst and fRefl >= fGood2:
            tmp = (worst, fWorst)
            worst, fWorst = refl, fRefl
            refl, fRefl = tmp
            contract()
        else:
            contract()

        perimeter = (worst - best).length() + (good2 - best).length() +
        (good1 - worst).length() + (good1 - good2).length() + (good2 -
        worst).length() + (best - good1).length()
        if perimeter < epsilon:
            break

```

```

        simplex = [ (worst, fWorst), (good2, fGood2), (good1, fGood1),
                    (best, fBest) ]

        return best

penaltyFunctionCalls = 0

def V(x, y, z):
    return -x * y * z

def Penalty(X):
    global r
    x, y, z = X.x, X.y, X.z
    return 1.0 / r * (max(0, -x)**2 + max(0, -y)**2 + max(0, -z)**2 +
(2*x*y + 2*x*z + 2*y*z - 1)**2)

def VPenalty(X):
    global penaltyFunctionCalls
    penaltyFunctionCalls += 1

    x, y, z = X.x, X.y, X.z

    return V(x, y, z) + Penalty(X)

def RunPenaltyAlgorithm(X):
    return simplexMethod(VPenalty, X)

if __name__ == '__main__':
    x0 = Vector(0.1, 0, 0.4)
    #x0 = Vector(1, 1, 1)
    #x0 = Vector(0, 0, 0)
    r = 10 #baudos kooficentas
    iterations = 0
    epsilon = 1e-6
    #prev_x0 = Vector(2, 2, 2)
    prevPenalty = Penalty(x0) + 0.001
    currentPenalty = Penalty(x0)

    while prevPenalty - currentPenalty > 0 or x0.x < 0 or x0.y < 0 or
x0.z < 0:
        #while abs(prevPenalty - currentPenalty) > epsilon:
        #while abs(x0.x - prev_x0.x) > epsilon or abs(x0.y - prev_x0.y) >
epsilon or abs(x0.z - prev_x0.z) > epsilon:
            B_value = VPenalty(x0)
            #print(f"r: {r}, (x, y, z): {x0}, Penalty: {currentPenalty}, V(X)
= {V(x0.x, x0.y, x0.z)}")
            print(f"r: {r}, (x, y, z): ({x0.x:.8f}, {x0.y:.8f}, {x0.z:.8f}),
Penalty: {currentPenalty}, B(X, r) = {B_value:.6f}, V(X) = {V(x0.x, x0.y,
x0.z):.6f}")

            #prev_x0 = x0

            x0 = RunPenaltyAlgorithm(x0)

            prevPenalty = currentPenalty

```

```
currentPenalty = Penalty(x0)

r/= 2
iterations += 1

print(f"""
    X = {x0}
    V(X) = {V(x0.x, x0.y, x0.z)}
    iterations = {iterations}
    penalty function calls = {penaltyFunctionCalls}
    """)
```